Dino Game Bot

A Deep Dive into Playing a Game by Counting Pixels

Portfolio explainer: exhaustive walkthrough

What this document is

A complete explanation of `dino-game-bot`: 812 lines of Python across three files that play Chrome's offline dinosaur game by taking screenshots and counting coloured pixels. It assumes you have never automated a browser, never written a screen-capture program, and do not know what an RGB value is. Every term is defined the first time it appears.
It is also honest. The project's own headline numbers reproduce exactly when you run them, which is more than most code manages. But running the code also turned up four things reading it would not have: the benchmark that justifies the project's main improvement is measuring less than it claims, the improvement it recommends adopting is roughly nine times slower than the thing it replaces, and the loop's advertised timing is off by a factor of seven. Those are in Section 8, and they are the most useful part of this document.

Contents

1 What the project is

Chrome has a hidden game. When your internet connection drops, the browser's error page shows a small pixel dinosaur, and pressing the space bar starts an endless runner: the dinosaur runs to the right, cacti and birds come at it, and you press space to jump over them. Hit one and the run ends. You can reach it deliberately at the address `chrome://dino/` whether you are online or not.

This project writes a program that plays it. Not by cheating (it never reads the game's internal state or its code), but by doing what a human does: looking at the screen, noticing an obstacle is close, and pressing space.

The one-sentence architecture

Take a screenshot of one small rectangle of the screen just in front of the dinosaur, ask whether any pixel in it is the colour an obstacle would be, and if so press space. Repeat forever.

That is genuinely the whole idea. Everything else in the project exists to support it, to make it observable while it runs, or to test it in an environment where the real game is not available.

1.1 Why this is an interesting exercise

The README states the motivation, and it is a fair one: the Dino game has exactly one input and a tiny visual vocabulary, which makes it a small, self-contained target for practising *screen-based automation*.

Jargon: Screen-based automation (screen scraping)

Controlling a program by reading the pixels it draws and sending it fake keyboard or mouse input, rather than by calling its functions. It is what you fall back on when a program offers no other way in: no API, no data file, no accessible internals. It is fragile by nature, because the only contract is "the picture keeps looking the same"; if the program's window moves, resizes, or restyles, the automation breaks even though nothing about its logic was wrong.

Jargon: API

Application Programming Interface: a documented set of functions one program offers so other programs can drive it directly. The Dino game has none, which is precisely why this project reads pixels.

1.2 The whole project at a glance

```
dino-game-bot
├── main.py                          536 lines: detection, the loop, the dashboard, demo mode
├── tools/
│   ├── generate_demo_frames.py      120 lines: draws six fake screenshots
│   └── compare_detection_robustness.py   156 lines: the before/after benchmark
├── demo_frames/                     six generated PNGs, committed
├── docs/screenshots/                one image, for the portfolio website
├── requirements.txt                 three pinned dependencies
├── .env.example                     one optional variable
├── README.md                        362 lines
└── LICENSE                          PolyForm Strict 1.0.0: viewable, not reusable
```

Figure 1: Three Python files, 812 lines measured. `main.py` carries the project; the two tools exist to feed and test it.

One file doing four jobs

`main.py` is 536 lines holding detection logic, a live loop, a browser driver, two status-view class hierarchies, and an entire offline replay mode. Nothing in it is wrong, and at this size it stays readable, but the natural seams are visible and unused: detection, presentation, and driving are three separable concerns that all share one module. If the project grew, this is the first thing that would need splitting. It is worth saying so before an interviewer does.

2 How the bot sees: pixels and colour

Before any of the code makes sense, two ideas.

Jargon: Pixel and RGB

A screen is a grid of tiny squares called pixels. Each one's colour is stored as three numbers from 0 to 255: how much Red, how much Green, how much Blue. So `(255, 255, 255)` is all three at maximum, which is white; `(0, 0, 0)` is black; `(83, 83, 83)` is an equal, low amount of each, which is a dark grey. A colour is therefore just a triple of numbers, and asking "is this pixel dark grey?" is asking "does this triple equal `(83, 83, 83)`?".

Jargon: Bounding box (bbox)

A rectangle of the screen, given as four numbers: `(left, top, right, bottom)`, measured in pixels from the top-left corner of the display. Note that *down* is the positive direction for `top`/`bottom`, which is standard for screens and the opposite of school graph paper. `(380, 645, 950, 760)` means the rectangle starting 380 pixels from the left and 645 pixels down, ending 950 across and 760 down.

The project pins four constants at the top of `main.py`, and everything else follows from them:

```
OBSTACLE_AREA_COORDINATES = (380, 645, 950, 760)
GAME_AREA_COORDINATES = (20, 580, 800, 760)
```

```
3
4 DAY_GROUND_COLOR = (255, 255, 255)
5 NIGHT_GROUND_COLOR = (83, 83, 83)
```

Listing 1: The four constants the whole bot rests on

2.1 The two regions, drawn to scale

The two boxes overlap, and the shape of the overlap explains most of the design.

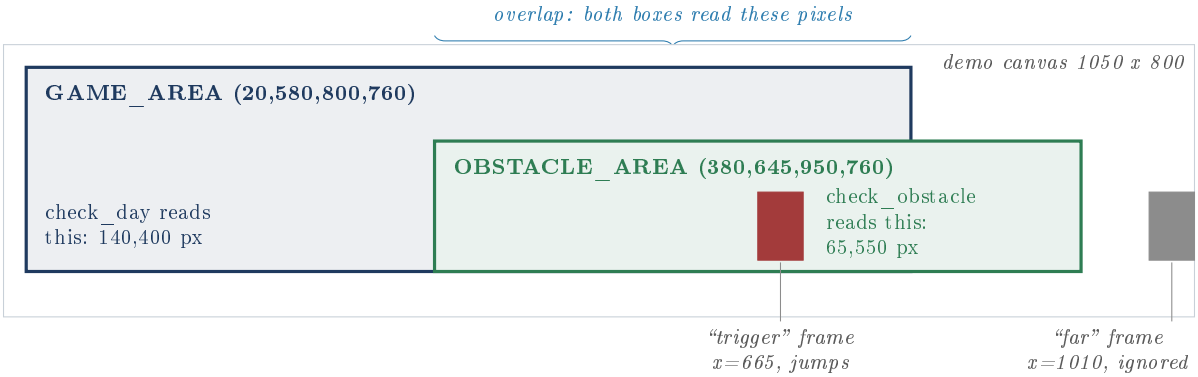

Figure 2: The two sampled regions in absolute screen coordinates, with the two obstacle positions the demo generator uses. Drawn to scale from the constants in `main.py` and `tools/generate_demo_frames.py`.

Three things to notice, because each one matters later:

- The obstacle box reaches further *right* (to 950) than the game box (800). It is looking further ahead down the track.
- The obstacle box is shorter: it starts at 645 rather than 580, so it ignores the upper part of the scene, which in a real game is sky.
- They share a bottom edge at 760, and they overlap across a wide band. A pixel at, say, `(500, 700)` is read twice per loop iteration, once by each function.

3 The detection layer

3.1 Why day and night both exist

The Dino game inverts its palette periodically: after a while the white background turns dark and the dark sprites turn white, then eventually back. This is the single fact that shapes the detection code.

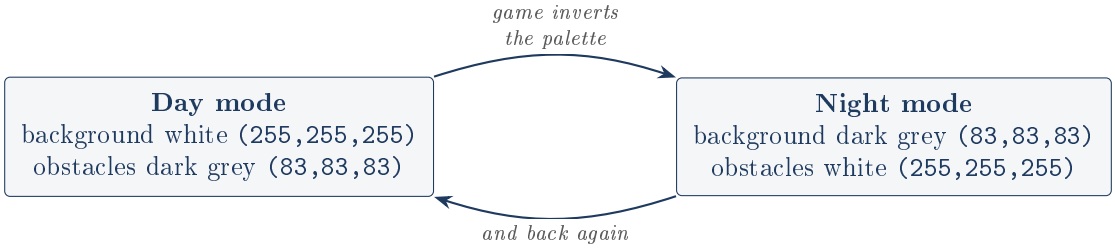

Figure 3: The two modes are exact colour swaps of each other. "What colour is an obstacle?" therefore has no fixed answer, which is why the mode must be established before the obstacle question can be asked.

Why `check_day` must run first

In day mode an obstacle is dark grey. In night mode an obstacle is white. A single fixed rule like "look for dark pixels" would be correct half the time and catastrophically wrong the other half, jumping at empty track or ignoring every cactus. So every obstacle check is preceded by a mode check, which picks which colour to hunt for. This costs a second screenshot on every iteration, and the project accepts that cost knowingly.

The constant names say "ground" and mean "background"

`DAY_GROUND_COLOR` is `(255, 255, 255)`, white. In the real game, white in day mode is the *sky and background*, not the ground; the ground line is dark. The names are describing the wrong thing, and the module docstring inherits the confusion, talking about a region containing "'ground' pixels" when it means the region contains *obstacle* pixels. `DAY_BACKGROUND_COLOR` would be accurate. This is cosmetic in the sense that the code works, and not cosmetic at all in the sense that a reader (including a future maintainer) has to hold a correction in their head the entire way through the file.

3.2 `check_day`: which mode are we in?

```
def check_day(tolerance=0):
    image = ImageGrab.grab(bbox=GAME_AREA_COORDINATES)
    pixels = list(image.getdata())
    if tolerance <= 0:
        day_count = pixels.count(DAY_GROUND_COLOR)
        night_count = pixels.count(NIGHT_GROUND_COLOR)
    else:
        day_count = sum(1 for p in pixels if _color_distance(p, DAY_GROUND_COLOR) <= tolerance
    )
        night_count = sum(1 for p in pixels if _color_distance(p, NIGHT_GROUND_COLOR) <=
    tolerance)
    return day_count > night_count
```

Listing 2: `check_day`, the exact-match path

Read it as four steps. Screenshot the large region. Flatten it into a flat list of 140,400 RGB triples with `getdata()`. Count how many are exactly white and how many are exactly dark grey. Return whether white won.

Jargon: `ImageGrab.grab`

A function from Pillow, the standard Python imaging library. Given a bbox it captures that rectangle of the live screen and hands back an image object. It is the bot's eye, and it is the single point the entire demo mode later hijacks.

The logic is a majority vote: whichever of the two known colours appears more often across a region covering both sky and ground tells you the palette. It is crude and it is cheap, and for a game with a two-colour palette it is genuinely sufficient.

A tie reports "night", confidently, on zero evidence

The return is `day_count > night_count`. If a frame contains neither colour, both counts are zero, `0 > 0` is `False`, and the function reports *night* with no evidence whatsoever for it. Verified by running it against a uniform mid-grey frame: it returns `False`. There is no third answer, no `None`, no "unsure". This matters more than it looks, and Section 8 shows it silently inflating the project's own benchmark.

3.3 `check_obstacle`: is something in the way?

```python
def check_obstacle(is_day=None, tolerance=0):
    image = ImageGrab.grab(bbox=OBSTACLE_AREA_COORDINATES)
    pixels = list(image.getdata())
    if is_day is None:
        is_day = check_day(tolerance=tolerance)
    target = NIGHT_GROUND_COLOR if is_day else DAY_GROUND_COLOR
    if tolerance <= 0:
        return target in pixels
    return any(_color_distance(p, target) <= tolerance for p in pixels)
```

Listing 3: `check_obstacle`, with both optional parameters

The key line is `target = NIGHT_GROUND_COLOR if is_day else DAY_GROUND_COLOR`, and it reads backwards until you see it: *in day mode, look for the night colour*. That is correct, because obstacles always render in the opposite colour to the background. In day mode obstacles are dark, and dark is `NIGHT_GROUND_COLOR`.

Then the test is deliberately weak: `target in pixels`. Not "how many", not "where", just *does this colour appear anywhere in the box*. One matching pixel out of 65,550 triggers a jump.

Presence is the whole algorithm

There is no distance estimate, no speed estimate, no sprite recognition, no tracking. The box is positioned close enough to the dinosaur that "an obstacle is inside this box" and "jump right now" are treated as the same statement. All the intelligence is in *where the box was drawn*, not in the code that reads it. That is why the coordinates are the most fragile thing in the project.

3.4 The `is_day` parameter is a screenshot-saving optimisation

`check_obstacle` can compute the mode itself, and if called bare it does. The optional `is_day` argument lets a caller that *already* knows the mode hand it over.

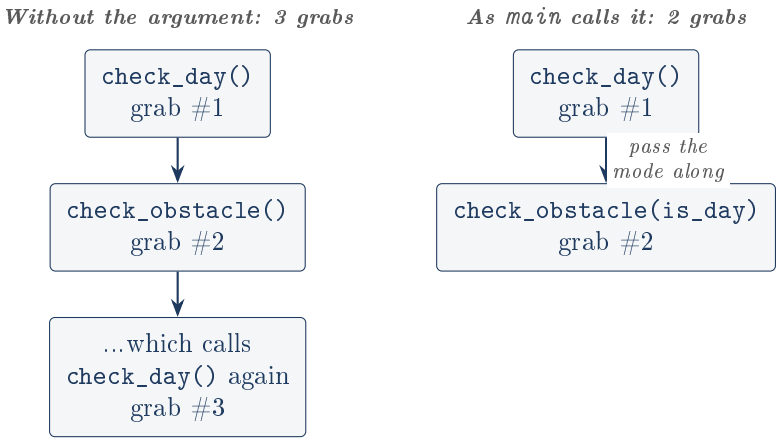

Figure 4: The `is_day` parameter removes one redundant screenshot and one 140,400-pixel scan per loop iteration. The claim in the README is accurate: this is a real saving, verified by reading both call paths.

3.5 `_color_distance` and the tolerance path

```python
def _color_distance(pixel, target):
    return max(abs(pixel[i] - target[i]) for i in range(3))
```

Listing 4: The tolerance helper

This returns the largest per-channel gap between two colours. If a pixel is `(250, 252, 249)` and the target is white `(255, 255, 255)`, the gaps are 5, 3 and 6, so the distance is 6. A pixel "matches" if that number is within `tolerance`.

Jargon: Chebyshev distance

Measuring the difference between two points as the largest difference along any one axis, rather than by the straight-line distance you would get from Pythagoras. It is cheaper to compute and, here, easier to reason about: "every channel is within 8" is a sentence you can hold in your head. The docstring calls it exactly that, "a cheap, easy-to-reason-about alternative to exact tuple equality".

Why it exists at all: an exact match demands a pixel be bit-for-bit `(255,255,255)`. Real screen captures do not oblige. Anti-aliasing softens sprite edges, dithering nudges values, compression rounds them. A pixel one unit off is, to an exact match, as wrong as black.

Off by default, on purpose

`tolerance` defaults to `0` in both functions, and `0` is a byte-for-byte reproduction of the original exact-match code. Nothing that currently runs passes a non-zero value except the benchmark. This is a deliberate and, in my view, correct decision: the feature is proven against synthetic noise only, so it is offered rather than imposed. The README says so plainly. Section 8 adds the cost figure that discussion is missing.

4 The decision loop

4.1 `main()` start to finish

```
def main():
    driver = build_driver()
    tracker = StatusTracker()
    status_view = build_status_view()

    try:
        driver.get("chrome://dino/")
    except WebDriverException:
        pass                                    # some drivers throw although the page loads
    driver.maximize_window()

    time.sleep(2)
    driver.find_element(value="t").send_keys(Keys.SPACE)    # start the game

    try:
        while True:
            is_day = check_day()
            obstacle = check_obstacle(is_day)
            jumped = False

            if obstacle:
                driver.find_element(value="t").send_keys(Keys.SPACE)
                jumped = True
                time.sleep(0.01)

            tracker.record(is_day, obstacle, jumped)
            status_view.update(tracker)

            if getattr(status_view, "closed", False):
                break
```

```
            if keyboard.is_pressed("q"):
                break
    finally:
        status_view.close()
        driver.quit()
```

Listing 5: The live loop, abridged to its skeleton

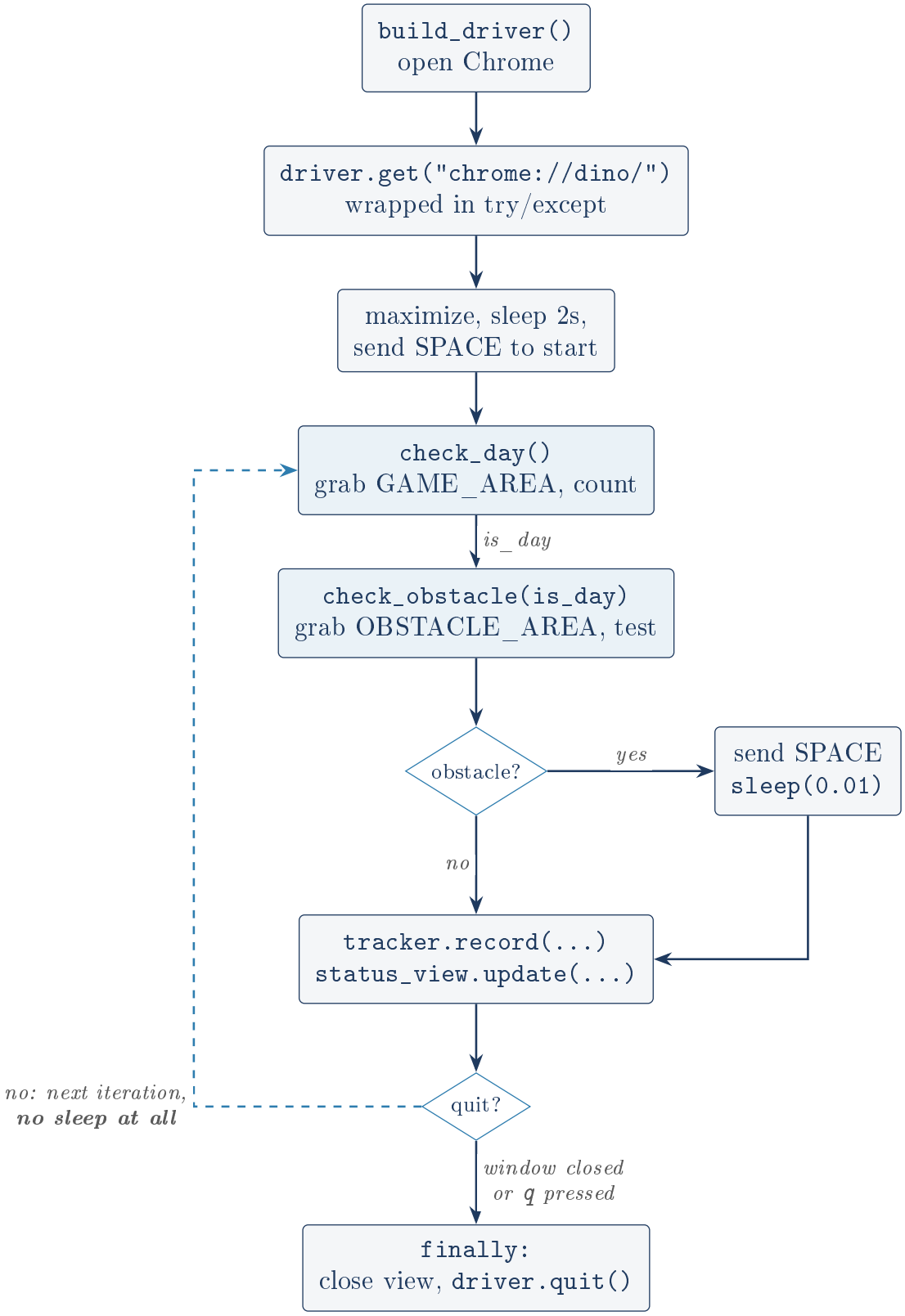

Figure 5: One pass of the live loop. The dashed return path is the hot path: on a clear track there is no sleep, so the loop spins as fast as the pixel scanning allows.

Jargon: `driver.find_element(value="t")`

Selenium's way of locating an element on the page; `"t"` is the id of the Dino game's canvas element. Sending a key to it delivers the keystroke to the game. Note this lookup is repeated

on *every* jump rather than cached once, a small avoidable cost inside the tightest part of the loop.

Jargon: Selenium and WebDriver

Selenium is a library for driving a real browser programmatically: open it, navigate, click, type. It talks to the browser through a helper program (`chromedriver` for Chrome). Since version 4.6 Selenium ships "Selenium Manager", which downloads a matching helper automatically, which is why this project needs no manual driver setup.

4.2 Two ways in, one way out

The loop can be stopped by closing the status window or by pressing `q`. The `try/finally` then guarantees the browser is shut down.

Why `finally` matters here

`driver.quit()` sits in a `finally` block, so it runs whether the loop exits normally, raises, or is interrupted. Without it, a stray exception would leave an orphaned Chrome process and a background `chromedriver` running with no owner. This is the difference between a script and a tool.

Headless live mode has no working stop button

The exit test is `getattr(status_view, "closed", False)`. `TkStatusView` defines a `closed` property; `ConsoleStatusView` does not define it at all, so `getattr` returns the `False` default forever. That is fine by itself, since there is no window to close. But it means that on a headless machine the *only* way out is `keyboard.is_pressed("q")`, and the README itself notes that on Linux the `keyboard` module needs root to read the keyboard device. So the configuration "Linux, no display, not running as root" has no in-band way to stop the loop at all. Nothing crashes; you just have to kill the process. Worth knowing before demonstrating it.

The `q` key is read globally, not from the browser

`keyboard.is_pressed("q")` hooks the operating system's keyboard, not Chrome's. It fires whether or not the browser has focus, which is convenient here and is also why the module wants root on Linux: reading the raw input device is a privileged operation. A reviewer may reasonably ask why a game bot needs `sudo`; this is the honest answer, and the honest follow-up is that Selenium could have supplied the quit key through the page instead, without privileges.

5 The status dashboard

The bot's problem is that it is invisible. Chrome is on screen, but nothing shows what the bot *believes*. The dashboard exists so the detection loop can be watched while it runs.

5.1 The split: state versus rendering

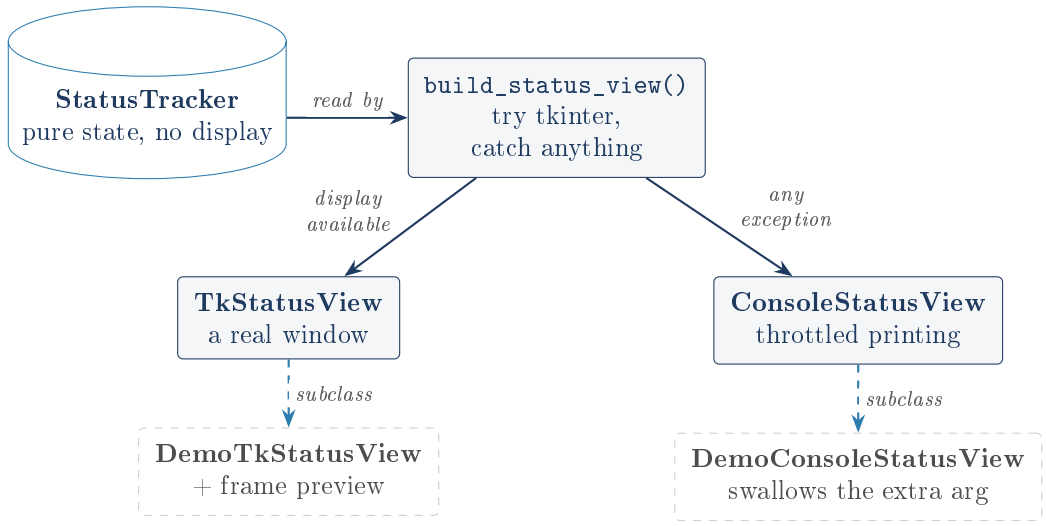

Figure 6: `StatusTracker` holds what the bot thinks; the views only render it. The demo variants extend the live views rather than modifying them, so live mode is untouched by demo mode's existence.

Why separating state from display is the right call

`StatusTracker` touches no window, no browser, no screen. It is a plain object you feed detection results to. That means its behaviour (mode flips, jump counting, log rollover) can be tested with no display and no Chrome at all, which is exactly what the README says was done. Every piece of logic you can pull out of a hard-to-test environment into a plain object is a piece you can actually verify.

5.2 `StatusTracker`

```
def record(self, is_day, obstacle_detected, jumped, timestamp=None):
    mode_changed = self.is_day is not None and is_day != self.is_day
    self.is_day = is_day
    self.obstacle_detected = obstacle_detected

    if jumped:
        self.jump_count += 1

    if timestamp is None:
        timestamp = datetime.now().strftime("%H:%M:%S")

    event = "jump" if jumped else ("obstacle" if obstacle_detected else "clear")
    if mode_changed:
        event += " (mode flip)"

    entry = {"time": timestamp, "mode": "day" if is_day else "night", "event": event}
    self.log.append(entry)
    return entry
```

Listing 6: `record`, called once per loop iteration

Three details worth pausing on.

- `mode_changed` guards on `self.is_day is not None`, so the very first iteration is never reported as a mode flip. Correct, and easy to get wrong.
- `log` is a `deque(maxlen=8)`.

- Both the clock and the timestamp are injectable (`clock=time.monotonic` in the constructor, `timestamp=None` here), purely so tests can pin them.

Jargon: `deque` with `maxlen`

A double-ended queue from Python's `collections`. Given a `maxlen`, it throws away the oldest item automatically when a new one would overflow it. That makes "keep the last 8 events" a data-structure choice rather than code you have to write and get wrong. Appending is constant time.

Jargon: `time.monotonic` versus `time.time`

`time.time` returns wall-clock time, which can jump backwards if the system clock is corrected or crosses a daylight-saving boundary. `time.monotonic` only ever moves forward, which is what you want for measuring elapsed time. Using it for `elapsed_seconds` is the right choice; using `datetime.now()` for the human-readable log timestamp is also right, because there you *want* wall-clock time.

5.3 `TkStatusView` and the blocking problem

Jargon: tkinter and `mainloop()`

tkinter is Python's built-in windowing toolkit. Normally you build a window and call `root.mainloop()`, which enters an infinite loop handling clicks, redraws, and keystrokes, and *never returns* until the window closes. That is fine when the GUI is the program.

Here the GUI is not the program: the detection loop is. Calling `mainloop()` would hand control to tkinter and the bot would stop playing. The project's answer:

```
    self.root.update_idletasks()
    self.root.update()
```

Listing 7: Pumping the event queue without surrendering control

Cooperative pumping instead of a blocking loop

`update_idletasks()` processes pending redraws; `update()` processes pending events. Together they do one slice of what `mainloop()` does forever, and then return immediately. The detection loop calls this once per iteration and keeps ownership of the thread. The window stays responsive, the bot keeps playing, and there are no threads and therefore no locks. For a loop that already runs continuously, this is a clean solution.

Closing the window sets `_closed` via the `WM_DELETE_WINDOW` protocol handler rather than destroying it mid-iteration, and the loop notices on its next pass. That ordering avoids the classic crash of a view being torn down while the loop still holds a reference to it.

5.4 `build_status_view`: degrade, do not die

```
def build_status_view(demo=False):
    tk_view_cls = DemoTkStatusView if demo else TkStatusView
    console_view_cls = DemoConsoleStatusView if demo else ConsoleStatusView
    try:
        return tk_view_cls()
    except Exception as exc:
        print(f"[status] no display available ({exc}); using console status output")
        return console_view_cls()
```

Listing 8: The fallback

A bare `except Exception` is usually a smell. Here it is defensible and I would defend it: the number of distinct ways constructing a Tk window can fail (no `DISPLAY`, Tk not compiled in, no X server, a container, a broken theme) is large, they raise different exception types, and the correct response to every one of them is identical. The failure is also reported rather than swallowed, and the fallback is strictly less capable rather than wrong.

The fallback catches too much, but only in theory

`except Exception` would also swallow a genuine bug inside `DemoTkStatusView.__init__` (say, a typo in a widget name) and quietly downgrade to console output, printing a misleading "no display available" message. The consequence is a confusing debugging session, not a wrong result. It is the right trade at this size; it would not be at ten times the size.

6 Demo mode: the most interesting idea in the project

The problem: this bot needs a live Chrome playing a live game, on a screen whose resolution matches the hardcoded constants. A fresh clone has none of that. So how does anyone, including the author, run the detection logic at all?

6.1 The trick

```
    original_grab = ImageGrab.grab
    try:
        for filename, frame in frames:
            ImageGrab.grab = lambda bbox=None, _frame=frame: _frame.crop(bbox)

            is_day = check_day()
            obstacle = check_obstacle(is_day)
            jumped = obstacle
            ...
    finally:
        ImageGrab.grab = original_grab
```

Listing 9: The heart of `run_demo`

Jargon: Monkeypatching

Replacing a function or attribute of an already-imported module at runtime, from outside it. Here `ImageGrab.grab`, the real screen-capture function, is swapped for a small stand-in that crops a pre-drawn image instead. Because `main.py` imported the *module* (`from PIL import ImageGrab`) and calls `ImageGrab.grab(...)` by attribute lookup at call time, the substitution is seen by every caller, with no cooperation from them.

Why this is better than an `if demo:` branch

The alternative would be a flag inside `check_day`/`check_obstacle` choosing between screen and file. That would mean demo mode tests a code path that live mode never takes, which is the exact thing that makes tests worthless. Patching the *eye* instead of the *brain* means `check_day` and `check_obstacle` are byte-for-byte the functions live mode runs; they cannot tell the difference and have no code that could. This is the project's best design decision.

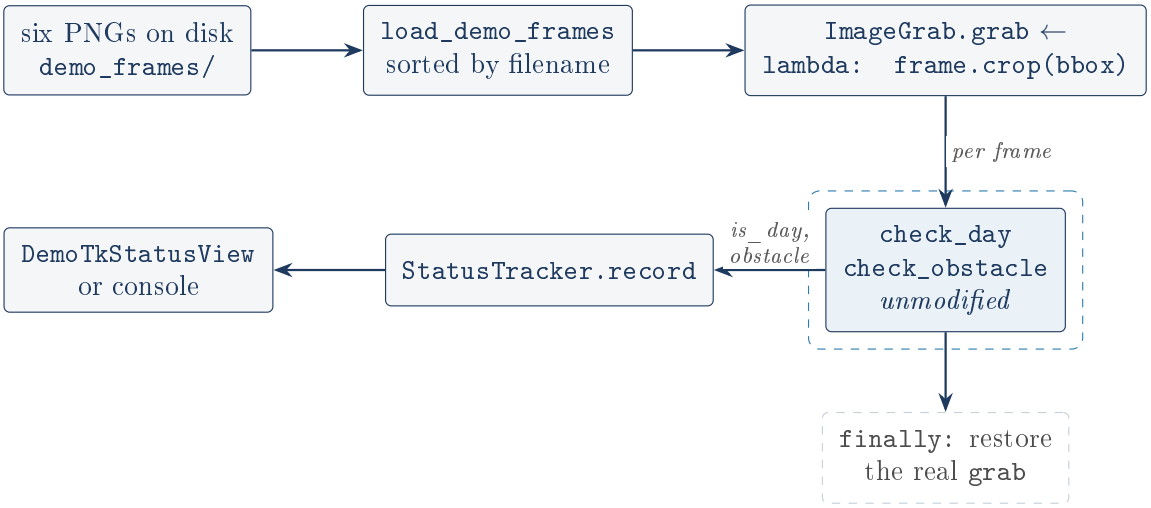

Figure 7: Demo mode's data flow. Only the boxed functions matter, and they are the same objects live mode calls. Everything else is scaffolding around them.

The `finally` restoring `original_grab` is not decoration. Without it, importing `main.py` and running a demo would leave `PIL.ImageGrab.grab` permanently replaced for the whole process, so any later real capture would return a stale dino frame. I verified the restore works: after `run_demo(delay=0)` returns, `ImageGrab.grab is` the original function.

6.2 `annotate_frame`: showing what the bot looked at

```
def annotate_frame(frame, obstacle_detected):
    annotated = frame.copy()
    draw = ImageDraw.Draw(annotated)
    draw.rectangle(GAME_AREA_COORDINATES, outline=GAME_AREA_BOX_COLOR, width=4)
    obstacle_color = OBSTACLE_BOX_COLOR_DETECTED if obstacle_detected else
    OBSTACLE_BOX_COLOR_CLEAR
    draw.rectangle(OBSTACLE_AREA_COORDINATES, outline=obstacle_color, width=4)
    return annotated
```

Listing 10: Drawing the sampled regions onto the preview

Blue for the region `check_day` read; green or red for the region `check_obstacle` read, red meaning it fired. The `frame.copy()` is load-bearing: without it the rectangles would be drawn permanently onto the loaded frame, and since the frames are held in memory across the replay loop, the boxes would accumulate. Small detail, correctly handled.

Jargon: Why `self._photo` is kept as an attribute

In `DemoTkStatusView.update`, the `PhotoImage` is stored on the instance with the comment "kept as an attribute so Tk doesn't garbage-collect it". This is a genuine tkinter trap: tkinter holds only a weak reference to image objects, so a `PhotoImage` kept solely in a local variable is collected the moment the function returns and the image silently renders blank. Assigning it to `self` keeps a strong reference alive. The author hit this and left a note; that note is worth more than the line it explains.

7 The two tools

7.1 `generate_demo_frames.py`

Draws the six frames. The design decision worth defending is at the top:

```python
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from main import (  # noqa: E402  (import after sys.path fixup, by design)
    DAY_GROUND_COLOR,
    GAME_AREA_COORDINATES,
    NIGHT_GROUND_COLOR,
    OBSTACLE_AREA_COORDINATES,
)
```

Listing 11: Importing the constants rather than copying them

One source of truth for the constants

The generator could have hardcoded `(83, 83, 83)` and `(380, 645, 950, 760)`. It imports them instead. So if someone re-measures the coordinates for their own screen, the demo frames follow automatically and cannot quietly drift out of sync with the code they are meant to test. A test fixture that duplicates the constants of the thing it tests is a test that eventually lies.

The six frames are systematic: two modes multiplied by three situations.

Frame	Mode	Obstacle at	Expected
`01_day_clear.png`	day	none	no jump
`02_day_obstacle_far.png`	day	x=1010 (outside)	no jump
`03_day_obstacle_trigger.png`	day	x=665 (inside)	**jump**
`04_night_clear.png`	night	none	no jump
`05_night_obstacle_far.png`	night	x=1010 (outside)	no jump
`06_night_obstacle_trigger.png`	night	x=665 (inside)	**jump**

Table 1: The frame set. The "far" frames are the interesting ones: an obstacle is visibly on screen but outside the trigger box, so a bot that jumped at them would be wrong. Positions computed by the generator from the real constants (`far_x = right + 60`, `inside_x = left + (right - left) // 2`).

I verified the frames on disk match this exactly. Each is 1050x800, and their colour histograms contain only the two expected values: `01_day_clear.png` is 840,000 pure white pixels and nothing else; `03_day_obstacle_trigger.png` is 837,499 white and 2,501 dark grey, which is the 41x61 rectangle the generator draws.

The frames omit the one feature most likely to break the bot

`make_frame` fills the entire canvas with a single flat colour. Its docstring is refreshingly upfront about this: "the real game has sky and a thin ground strip, but `check_day` only ever compares white-pixel count vs. grey-pixel count". That reasoning is sound *for* `check_day`. It does not hold for `check_obstacle`, which does not count anything: it fires if the target colour appears *even once* in its box. In the real day-mode game the ground line is dark grey, which is precisely the colour `check_obstacle` hunts for in day mode, and the obstacle box spans y=645 to y=760, which is the band where the ground line would plausibly sit given the dinosaur stands on it. If the ground line falls inside that box, `check_obstacle` returns `True` on every single frame and the bot jumps continuously forever. I want to be precise about the status of this: that the synthetic frames contain no ground strip is **confirmed** (I checked the histograms; two colours, no third). That the real game's ground line falls inside the box is **inferred**, and unverifiable here, because there is no Chrome and no live game in this environment. It may well be that the box was tuned to sit above the ground line, which would be the explanation for why it ever worked.

Either way the point stands: the demo set cannot distinguish those two worlds, so "all six frames pass" is much weaker evidence than it sounds. It is the exact class of bug a synthetic fixture is worst at catching, because the fixture was drawn by the same person holding the same assumption as the code.

7.2 `compare_detection_robustness.py`

This is the script that justifies the `tolerance` feature. Its shape is a proper experiment, and I want to give it credit before criticising it:

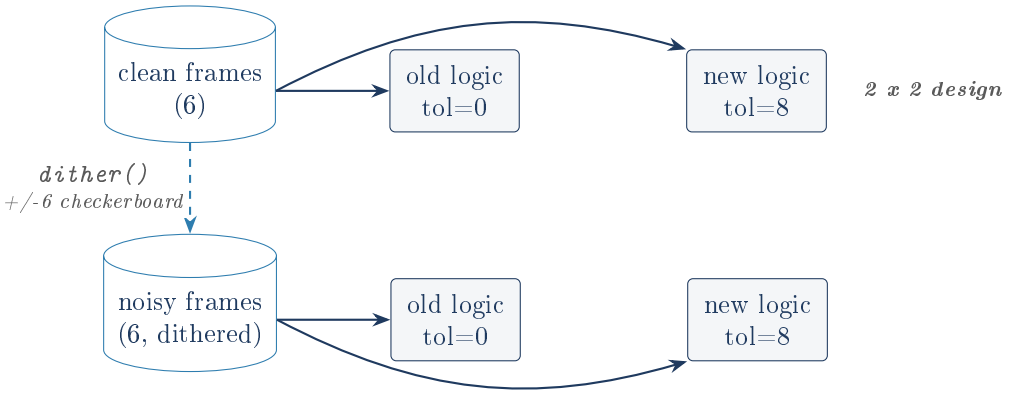

Figure 8: The benchmark runs both logics against both frame sets: a two-by-two comparison, so "did it help" and "did it break anything" are answered by the same run.

Two questions, not one

The clean row asks "does the new logic regress what already worked?". The noisy row asks "does it fix what did not?". A change that only answers the second is not evidence. The script also derives the expected answer from each frame's *filename* rather than a hand-written table, so the ground truth cannot drift out of sync either. And it prints an explicit verdict, refusing to endorse the change unless clean is not regressed *and* noisy improved.

I ran it. The README's numbers reproduce exactly:

	clean frames	noisy frames
old (`tolerance=0`)	6/6 correct	4/6 correct
new (`tolerance=8`)	6/6 correct	6/6 correct

Table 2: Reproduced by running `python3 tools/compare_detection_robustness.py` in this environment. The README's table is accurate, and the two old failures are the ones it names: a missed obstacle on `03`, and night misread as day on `06`.

That is a real, checkable claim that survived checking, which is worth saying plainly. The problem is not the numbers. It is what they measure.

8 Honest findings

Everything in this section is from running the code in this environment, not from reading it.

8.1 The "noisy" frames are not noisy where it matters

`dither()` claims, in its own docstring, that it "guarantees no pixel in the output is ever exactly equal to a color present in the input". That guarantee is false, and it fails in the direction that flatters the benchmark.

```
out_pixels[x, y] = (
    max(0, min(255, r + sign)),
    ...
)
```

Listing 12: The clipping bug, in the arithmetic

For a dark grey channel, 83: `83 + 6 = 89`, `83 - 6 = 77`. Neither is 83, so grey is genuinely perturbed. For a white channel, 255: `255 - 6 = 249`, but `255 + 6 = 261`, which `min(255, ...)` clips straight back to **255**. Half the white pixels, the ones on the `+` squares of the checkerboard, come through the "noise" completely untouched.

I measured the resulting histograms:

Noisy frame	exact white survivors	exact grey survivors
`01_day_clear.png`	420,000	0
`04_night_clear.png`	0	0
`06_night_obstacle_trigger.png`	1,250	0

Table 3: Measured by applying `dither()` and counting colours. On the day frame, exactly half the pixels survive the noise bit-for-bit intact.

The benchmark is asymmetric, and it was never meant to be

The noise fully destroys grey and only half-destroys white. So on the noisy day frames the old exact-match logic still finds 420,000 perfect white pixels and sails through `check_day`, not because exact matching is robust, but because the noise generator could not push 255 any higher. Meanwhile the same generator moves every grey pixel off its exact value, which is why the old logic fails `03`'s obstacle check. The experiment is not comparing exact-vs-tolerant under noise; it is comparing them under noise that only applies to one of the two colours. Fixing it is a one-line change (subtract when the channel is near the ceiling, or dither around a midpoint), and the honest expectation is that the old logic's noisy score would drop below 4/6, making the tolerance feature look *better*, not worse. The conclusion probably survives. The measurement does not support it as it stands.

8.2 Two of the old logic's four passes are worth nothing

Recall that `check_day` returns `day_count > night_count`, so a tie reports night. On the noisy `04_night_clear.png`, both counts are zero: no pixel is exactly white and none is exactly grey. The old logic therefore reports "night" having seen no evidence at all, the frame happens to be a night frame, and the benchmark scores it **correct**. The same applies to `05_night_obstacle_far.png`.

4/6 flatters the old logic; its real score is 2/6

Of the old logic's four correct answers on the noisy set, two (`04`, `05`) are the zero-evidence tie defaulting to night and getting lucky, and one (`01`) is the clipping artefact above. Only `02` is arguably a genuine pass, and it inherits the same clipping. If the night frames in the set had been day frames, the old logic would have scored them wrong for exactly the same reason it scored them right. A benchmark that cannot tell "correct" from "no opinion, defaulted, guessed right" is measuring the wrong thing. This is not an argument against the tolerance

feature, which I think is correct. It is an argument that the evidence offered for it is softer than the table implies, and the fix is to make `check_day` report "unknown" on a tie instead of silently voting night.

8.3 The tolerance path costs nine times more, and nobody measured it

This is the finding with the most practical consequence. The README's "What I'd do differently" recommends turning tolerance on by default once it has been checked against a real capture. Nowhere does it mention what that would cost. I measured it, on this machine, with the real screen grab removed so only the pixel work is counted:

Call	tolerance=0	tolerance=8
`check_day` (140,400 px)	50.3 ms	531.8 ms
`check_obstacle` (65,550 px)	19.7 ms	76.5 ms
per loop iteration	**70.0 ms**	**608.3 ms**
implied loop rate	**14 Hz**	**1.6 Hz**

Table 4: Measured over 20 calls each against a demo frame, in this environment, excluding the real `ImageGrab.grab` (which would add more). The ratio, not the absolute figure, is the portable part.

The cause is not subtle. At `tolerance=0`, `pixels.count(target)` and `target in pixels` are implemented in C and scan the list at C speed. At `tolerance > 0`, both become Python-level generator expressions calling `_color_distance` once per pixel, and `_color_distance` itself builds a generator and calls `max`, `abs` and three subscripts. That is roughly 140,000 Python function calls where there used to be one C loop.

The recommended default would take the bot from 14 Hz to 1.6 Hz

Adopting `tolerance=8` by default, as the README proposes, would slow each decision from about 70 ms to about 600 ms of pure pixel arithmetic. A dino game obstacle crosses the trigger box in a fraction of a second at speed. A bot that forms an opinion 1.6 times a second will not clear obstacles; it will hit them while still counting pixels from a frame that is already stale. The feature is correct and the benchmark endorses its *accuracy*, but accuracy was the only axis measured, and for a real-time loop it is not the only axis that decides. This is fixable and not hard (do the tolerance comparison with Pillow's `point()`/`ImageChops` on the whole image in C, or downsample the region first, or drop from 140,400 pixels to a sparse grid of a few hundred), but as written, "turn it on by default" would break the bot. Nothing in the repository says so.

8.4 The 10 ms sleep is a rounding error

The README lists "Reaction time vs. polling cost" as a challenge and describes the 10 ms sleep as "trading CPU usage for faster reaction time". The measurement above shows the sleep is not the term that matters: 10 ms sits next to roughly 70 ms of unavoidable pixel scanning, so it accounts for about one seventh of the iteration and only on iterations that actually jumped.

The loop is a full-speed busy wait, and its real period is not 10 ms

Two separate points. First, `time.sleep(0.01)` is *inside* `if obstacle:`, so on a clear track the loop never sleeps at all: it pins one CPU core at 100% indefinitely, taking two screenshots and scanning 206,000 pixels per pass, forever. Second, the mental model the README implies

(a ∼10 ms polling loop, so ∼100 checks a second) is off by roughly seven times: measured, the detection work alone caps the loop near 14 Hz before the real screen grab is even counted. Tuning that sleep, in either direction, would change almost nothing. The lever that would matter is the pixel count: `check_day` scans 140,400 pixels to answer a single binary question that a 200-pixel sample would answer just as well, and it does it on every iteration although the palette flips maybe once a minute. Caching the mode and rechecking it a few times a second would cut roughly 70% of the loop's cost for no loss.

8.5 Smaller things

The README's "Challenges" section documents the wrong project

Two of the eight bullets under "Challenges" are about LaTeX: one about a `tikz` node needing `align=center`, one about `tcolorbox` option parsing choking on a comma inside a title. Both are real problems, and neither is a challenge of *this project*; they are challenges of the toolchain that built this project's explainer PDFs. A reader arriving at a screen-automation bot's README to find it discussing pdflatex error messages will be confused, and an interviewer asking "tell me about the hardest part of the dino bot" will not want to hear about tcolorbox. They belong in the explainer toolchain's own notes.

`find_element` is re-queried on every jump

`driver.find_element(value="t")` runs inside the `if obstacle:` branch, so every jump pays a fresh round trip to chromedriver to look up an element that never changes. Hoisting it to a variable before the loop would remove a network hop from the latency-critical path. Minor next to the 70 ms of pixel work, but it is free to fix.

`DemoConsoleStatusView` exists only to swallow an argument

Its entire body is `def update(self, tracker, frame_image=None): super().update(tracker)`. It exists so `run_demo` can call `status_view.update(tracker, frame_image=annotated)` uniformly against either view. That is a legitimate reason, and it is honestly documented. It is still a class whose only purpose is to discard a parameter, and a default argument on `ConsoleStatusView.update` would have done the same job without the subclass.

9 Dependencies and licence

Package	Pinned	Role
`selenium`	4.24.0	Opens Chrome, navigates, delivers the space keypress
`Pillow`	10.4.0	`ImageGrab` (the eye), `Image`/`ImageDraw` (demo frames)
`keyboard`	0.13.5	Global `q` hotkey; needs root on Linux

Table 5: Three dependencies, all pinned to exact versions. tkinter is not listed because it ships with Python.

Why exact pins

`==` rather than `>=` means a clone a year from now installs the same versions this was verified against. For a project whose correctness depends on screen-capture behaviour, that is the right call.

`CHROMEDRIVER_PATH` is the only environment variable, and it is optional: `build_driver` falls back to letting Selenium Manager locate a driver. The README notes this replaced a hard-coded Windows path (`C:\Development\chromedriver.exe`) from the original version, which is a real portability fix.

The licence is PolyForm Strict 1.0.0: the code may be read but not reused or redistributed. Appropriate for a portfolio piece meant to be inspected rather than adopted.

10 What is verified, and what is not

Verified by me, in this environment (a display was available; no Chrome, no live game):

- `python3 main.py --demo` runs to completion and produces the correct decision on all six frames: day/night read correctly, jump fired only on `03` and `06`.
- The same run with `DISPLAY` unset falls back to the console view, prints "no display available", and produces identical per-frame decisions.
- `ImageGrab.grab` is restored to the real function after `run_demo` returns.
- `tools/compare_detection_robustness.py` reproduces the README's 6/6, 6/6, 4/6, 6/6 table exactly, including which two frames fail.
- The six committed PNGs are 1050x800 and contain only the two expected colours, in the pixel counts the generator's geometry predicts.
- `check_day` returns `False` (night) on a frame containing neither target colour.
- `dither()` leaves 420,000 pixels of `01_day_clear.png` exactly white, contradicting its own docstring.
- The timing table in Section 8, measured over 20 calls each.

Not verified, and not verifiable without a real machine running the game:

- That Chrome opens, `chrome://dino/` loads, and the game responds to the keypress at all.
- Whether `OBSTACLE_AREA_COORDINATES` and `GAME_AREA_COORDINATES` match any real dino window today. They are almost certainly wrong for any screen other than the one they were measured on.
- Whether the real day-mode ground line falls inside the obstacle box and causes the permanent-jump failure inferred above.
- Whether the bot actually clears obstacles over a real run.
- Whether tolerance-based matching helps against real capture noise, as opposed to the synthetic checkerboard analysed above.

The honest summary

This is a small project that does one thing and is unusually well-instrumented for its size. Its best ideas are real: patching the eye rather than branching the brain so demo mode exercises production code, importing constants into the fixture so it cannot drift, and refusing to adopt a change the benchmark did not endorse. Its weakness is that the benchmark it trusts measures accuracy on frames whose noise is half-broken, ignores speed entirely, and cannot see the failure mode most likely to be real. None of that is hard to fix, and all of it is better to know first.